Trabalho Computacional 1 – MS211

Amanda Rehder Cedro – RA 253507

2025-09-30

Table of contents

1 Introdução: **2**

2 Questão 1 – Aritmética de Ponto Flutuante e Erros **2**
2.1 Explicaç]ão do conteúdo: . . . 2
2.2 Código . . . 2
2.3 Justificativa para a diferença de erros: . . . 3

3 Questão 2 – Zeros de Funções Reais **3**
3.1 Newton-Raphson . . . 4
3.2 Secante . . . 5
3.3 Bissecção . . . 9
3.4 Comparação e Discussão . . . 11

4 Questão 3 – Sistemas Lineares **11**
4.1 Eliminação de Gauss . . . 11
4.2 Matrizes de Hilbert . . . 15
4.3 Gauss-Jacobi . . . 17
4.4 Discussão . . . 22

5 Questão 4 – Sistemas Não Lineares **22**
5.1 Implementação do Método de Newton . . . 22
5.2 Resultados e Gráficos . . . 24

6 Conclusão **30**

7 Bibliografia **30**

1 Introdução:

O trabalho tem como objetivo desenvolver competências em implementação numérica, compreender fundamentos teóricos dos métodos numéricos e desenvolver a capacidasde de análise crítica a partir da criação de códigos para as questões e a análise dos resultados. Para que isso fosse possível, foi utilizado o Software RStudio com o pacote `ggplot2` para a implementação de gráfico.

2 Questão 1 – Aritmética de Ponto Flutuante e Erros

2.1 Explicaç]ão do conteúdo:

A aritmética de ponto flutuante é um sistema para representar números reais em computadores, que possuem precisão finita. Diferente da matemática exata, os computadores trabalham com representações aproximadas devido às limitações de memória. Como estamos trabalhando com máquinas computacionais, números com representação exata finita em decimal podem ter representação infinita em binário, o que gera erros de arredendamento na implementação de contas, assim como veremos a seguir.Também pode ocorrer um cancelamento quando subtraimos números muito pequenos e próximos. Outros erros que podem acontecer são os chamados overflow e underflow: **Overflow**: Número muito grande para ser representado **Underflow**: Número muito próximo de zero

2.2 Código

```r
# Questão 1
options(digits = 15)

n <- 30000

soma_parcela <- function(xi) {
  S_exato <- n * xi
  S_acumulado <- 0
  for (i in 1:n) {
    S_acumulado <- S_acumulado + xi
  }
  erro_abs <- abs(S_acumulado - S_exato)
  erro_rel <- erro_abs / abs(S_exato)
  cat("\n=== Caso xi =", xi, "===\n")
  cat("S_exato      =", format(S_exato, scientific = FALSE), "\n")
```

```r
  cat("S_acumulado  =", format(S_acumulado, scientific = FALSE), "\n")
  cat("Erro Absoluto=", format(erro_abs, scientific = TRUE), "\n")
  cat("Erro Relativo=", format(erro_rel, scientific = TRUE), "\n")
}

soma_parcela(0.5)
```

```
=== Caso xi = 0.5 ===
S_exato      = 15000
S_acumulado  = 15000
Erro Absoluto= 0e+00
Erro Relativo= 0e+00
```

```r
soma_parcela(0.11)
```

```
=== Caso xi = 0.11 ===
S_exato      = 3300
S_acumulado  = 3300.00000000063
Erro Absoluto= 6.28460838925093e-10
Erro Relativo= 1.9044267846215e-13
```

2.3 Justificativa para a diferença de erros:

A partir desse código é possível ver que para xi= 0.5, um número que pode ser representado de forma exata no formato binário, não apresenta erro significativo conforme vamos realizando as somas, ao contrário do que acontece com xi= 0.11, que não pode ser apresentado de forma finita no formato binário, assim, os erros vão se acumulando com as somas e se tornam significativos, assim, podemos ver que, para xi= 0.11, os resultados da soma exata e da soma parcelada são diferentes, já que acontece um arredondamento de valores.

3 Questão 2 – Zeros de Funções Reais

Encontrar os zeros (ou raízes) de uma função f(x) significa determinar os valores de x tal quef(x)=0. Métodos iterativos são necessários quando não existe solução analítica.

3.1 Newton-Raphson

```
# Implementação do método
# Questão 2(a)
f  <- function(x) x^3 - 9*x + 3
df <- function(x) 3*x^2 - 9

x  <- 0.5
E1 <- 1e-4
E2 <- 1e-4
maxit <- 10000

cat(sprintf("%-4s | %-14s | %-14s | %-14s | %-10s\n",
            "Iter", "xk", "f(xk)", "f'(xk)", "|Δx|/|xk|"))
```

```
Iter | xk             | f(xk)          | f'(xk)         | |Δx|/|xk|
```

```
cat(rep("-", 70), "\n", sep = "")
```

```
----------------------------------------------------------------------
```

```
fx <- f(x)
cat(sprintf("%3d  | %14.12f | %14.6e | %14.12f |     ---\n", 0, x, fx, df(x)))
```

```
  0  | 0.500000000000 |  -1.375000e+00 | -8.250000000000 |     ---
```

```
for (k in 1:maxit) {
  dfx <- df(x)
  if (dfx == 0) stop("Derivada zero, método não pode prosseguir")
  x_novo <- x - fx/dfx
  fx_novo <- f(x_novo)
  delta_rel <- abs(x_novo - x) / max(abs(x_novo), .Machine$double.eps)

  cat(sprintf("%3d  | %14.12f | %14.6e | %14.12f | %10.5e\n",
              k, x_novo, fx_novo, df(x_novo), delta_rel))

  if (abs(fx_novo) < E1 && delta_rel < E2) {
    cat(sprintf(">> Critérios satisfeitos. Método parou na iteração %d.\n", k))
    break
```

```
  }

  x <- x_novo
  fx <- fx_novo
}
```

```
  1   | 0.333333333333 |   3.703704e-02 | -8.666666666667 | 5.00000e-01
  2   | 0.337606837607 |   1.834089e-05 | -8.658064869603 | 1.26582e-02
  3   | 0.337608955965 |   4.544809e-12 | -8.658060578556 | 6.27459e-06
>> Critérios satisfeitos. Método parou na iteração 3.
```

```
cat(sprintf("\nRaiz aproximada: x = %.12f\nf(x) = %.3e\n", x_novo, fx_novo))
```

```
Raiz aproximada: x = 0.337608955965
f(x) = 4.545e-12
```

3.2 Secante

```
# Implementação do método da Secante com raiz final impressa
f <- function(x) x^3 - 9*x + 3

x0 <- 0; x1 <- 1
E1 <- 5e-4; E2 <- 5e-4
maxit <- 10000

cat(sprintf("%-4s | %-14s | %-14s | %-10s\n", "Iter", "x_k", "f(x_k)", "|Δx|/|x_k|"))
```

```
Iter | x_k            | f(x_k)         | |Δx|/|x_k|
```

```
cat(rep("-",60), "\n", sep="")
```

```
------------------------------------------------------------
```

```
cat(sprintf("%3d  | %14.12f | %14.6e |     ---\n", 0, x0, f(x0)))
```

```
  0   | 0.000000000000 |   3.000000e+00 |     ---
```

```r
cat(sprintf("%3d  | %14.12f | %14.6e | %10.5e\n", 1, x1, f(x1), abs(x1-x0)/abs(x1)))
```

```
  1  | 1.000000000000 |  -5.000000e+00 | 1.00000e+00
```

```r
raiz_final <- NA  # variável para guardar a raiz

for (k in 2:maxit) {
  f0 <- f(x0); f1 <- f(x1)
  denom <- f1 - f0
  if (denom == 0) stop("denominador zero")

  x2 <- x1 - f1*(x1 - x0)/denom
  fx2 <- f(x2)
  delta_rel <- abs(x2 - x1) / max(abs(x2), .Machine$double.eps)

  cat(sprintf("%3d  | %14.12f | %14.6e | %10.5e\n", k, x2, fx2, delta_rel))

  if (abs(fx2) < E1 && delta_rel < E2) {
    raiz_final <- x2
    cat(sprintf(">> Parou em it=%d\n", k))
    break
  }

  x0 <- x1; x1 <- x2
}
```

```
  2  | 0.375000000000 |  -3.222656e-01 | 1.66667e+00
  3  | 0.331941544885 |   4.910114e-02 | 1.29717e-01
  4  | 0.337634620723 |  -2.222064e-04 | 1.68616e-02
  5  | 0.337608972875 |  -1.464017e-07 | 7.59691e-05
>> Parou em it=5
```

```r
if (!is.na(raiz_final)) {
  cat(sprintf("\nRaiz aproximada encontrada: %.12f\n", raiz_final))
  cat(sprintf("f(raiz) = %.3e\n", f(raiz_final)))
}
```

```
Raiz aproximada encontrada: 0.337608972875
f(raiz) = -1.464e-07
```

```r
secante <- function(f, a, b, n=100, tol=1e-2, E1=5e-4, E2=5e-4, maxit=20000) {
  x_seq <- seq(a, b, length.out = n+1)
  f_seq <- f(x_seq)

  intervalos <- list()
  for (i in 1:n) {
    if (f_seq[i]*f_seq[i+1] < 0) {
      intervalos[[length(intervalos)+1]] <- c(x_seq[i], x_seq[i+1])
    }
    if (abs(f_seq[i]) < tol) {
      intervalo_local <- c(x_seq[i]-abs(x_seq[i+1]-x_seq[i])/2,
                           x_seq[i]+abs(x_seq[i+1]-x_seq[i])/2)
      intervalos[[length(intervalos)+1]] <- intervalo_local
    }
  }

  if (length(intervalos)==0) stop("Nenhum intervalo candidato encontrado.")
  intervalos <- unique(intervalos)

  resultados <- list()

  for (idx in seq_along(intervalos)) {
    int <- intervalos[[idx]]
    x0 <- int[1]; x1 <- int[2]
    f0 <- f(x0); f1 <- f(x1)
    if (f0*f1 > 0) next

    # Guardar iterações em data.frame
    iteracoes <- data.frame(iter=0, x=x0, fx=f0)
    iteracoes <- rbind(iteracoes, data.frame(iter=1, x=x1, fx=f1))

    for (k in 2:maxit) {
      denom <- f1 - f0
      if (abs(denom) < .Machine$double.eps) break

      x2 <- x1 - f1 * (x1 - x0) / denom
      fx2 <- f(x2)

      iteracoes <- rbind(iteracoes, data.frame(iter=k, x=x2, fx=fx2))

      delta_rel <- abs(x2 - x1) / max(abs(x2), .Machine$double.eps)
      if (abs(fx2) < E1 && delta_rel < E2) {
```

```r
        resultados[[paste0("raiz_", idx)]] <- list(
          intervalo = int,
          raiz = x2,
          iteracoes = iteracoes
        )
        break
      }

      x0 <- x1; x1 <- x2
      f0 <- f1; f1 <- fx2
    }
  }

  return(resultados)
}

# Teste
f <- function(x) x^3 - 9*x + 3
raizes <- secante(f, -10, 10, n=2000)

# Mostrar iterações da primeira raiz encontrada
print(raizes)
```

```
$raiz_1
$raiz_1$intervalo
[1] -3.16 -3.15

$raiz_1$raiz
[1] -3.15452303180037

$raiz_1$iteracoes
  iter                 x                    fx
1    0 -3.16000000000000 -1.14496000000003e-01
2    1 -3.15000000000000  9.41250000000089e-02
3    2 -3.15451177014778  2.34356753832543e-04
4    3 -3.15452303180037 -4.81813081876226e-07


$raiz_2
$raiz_2$intervalo
[1] 0.33 0.34
```

```
$raiz_2$raiz
[1] 0.337608955372407

$raiz_2$iteracoes
  iter                 x                     fx
1    0 0.330000000000000  6.59369999999995e-02
2    1 0.340000000000000 -2.06959999999987e-02
3    2 0.337611071993351 -1.83206898629606e-05
4    3 0.337608955372407  5.13796294399071e-09


$raiz_3
$raiz_3$intervalo
[1] 2.81 2.82

$raiz_3$raiz
[1] 2.81691403126934

$raiz_3$iteracoes
  iter                x                     fx
1    0 2.81000000000000 -1.01958999999994e-01
2    1 2.82000000000000  4.57680000000025e-02
3    2 2.81690185274188 -1.80619731903420e-04
4    3 2.81691403126934 -3.17715997510959e-07
```

3.3 Bissecção

```r
# Implementação do método
bisseccao <- function(f, a, b, E1=5e-4, E2=5e-4, maxit=1000) {
  if (f(a) * f(b) > 0) stop("Não há garantia de raiz: f(a) e f(b) têm o mesmo sinal!")

  cat(sprintf("%-4s | %-14s | %-14s | %-10s\n", "Iter", "x_mid", "f(x_mid)", "Erro Relativo"))
  cat(rep("-",60), "\n", sep="")

  raiz_final <- NA

  for (k in 1:maxit) {
    c <- (a + b)/2
    fc <- f(c)
    erro_rel <- abs(b - a) / max(abs(c), .Machine$double.eps)
```

```r
    cat(sprintf("%3d  | %14.12f | %14.6e | %10.5e\n", k, c, fc, erro_rel))

    if (abs(fc) < E1 && erro_rel < E2) {
      raiz_final <- c
      cat(sprintf(">> Convergiu na iteração %d\n", k))
      break
    }

    # Reduzir intervalo
    if (f(a)*fc < 0) {
      b <- c
    } else {
      a <- c
    }
  }

  if (!is.na(raiz_final)) {
    cat(sprintf("\nRaiz aproximada encontrada: %.12f\n", raiz_final))
    cat(sprintf("f(raiz) = %.3e\n", f(raiz_final)))
  } else {
    cat(">> Número máximo de iterações atingido sem convergência.\n")
  }

  return(invisible(raiz_final))
}

# --- TESTE ---
f <- function(x) x^3 - 9*x + 3
raiz_bisseccao <- bisseccao(f, 0, 1, E1=5e-4, E2=5e-4)
```

```
Iter | x_mid          | f(x_mid)       | Erro Relativo
-------------------------------------------------------------
  1  | 0.500000000000 |  -1.375000e+00 | 2.00000e+00
  2  | 0.250000000000 |   7.656250e-01 | 2.00000e+00
  3  | 0.375000000000 |  -3.222656e-01 | 6.66667e-01
  4  | 0.312500000000 |   2.180176e-01 | 4.00000e-01
  5  | 0.343750000000 |  -5.313110e-02 | 1.81818e-01
  6  | 0.328125000000 |   8.220291e-02 | 9.52381e-02
  7  | 0.335937500000 |   1.447439e-02 | 4.65116e-02
  8  | 0.339843750000 |  -1.934391e-02 | 2.29885e-02
  9  | 0.337890625000 |  -2.438627e-03 | 1.15607e-02
 10  | 0.336914062500 |   6.016918e-03 | 5.79710e-03
```

```
 11  | 0.337402343750 |   1.788904e-03 | 2.89436e-03
 12  | 0.337646484375 |  -3.249218e-04 | 1.44613e-03
 13  | 0.337524414062 |   7.319762e-04 | 7.23327e-04
 14  | 0.337585449219 |   2.035234e-04 | 3.61598e-04
>> Convergiu na iteração 14

Raiz aproximada encontrada: 0.337585449219
f(raiz) = 2.035e-04
```

3.4 Comparação e Discussão

Analise número de iterações e convergência.

4 Questão 3 – Sistemas Lineares

4.1 Eliminação de Gauss

```r
# Implementação com pivoteamento parcial
gauss_parcial <- function(A, b, verbose = TRUE) {
  if (!requireNamespace("MASS", quietly = TRUE)) {
    stop("Por favor instale o pacote MASS: install.packages('MASS')")
  }

  n <- nrow(A)
  Aug <- cbind(A, b)   # matriz aumentada (n x (n+1))

  pretty_print <- function(M) {
    print(MASS::fractions(M))
  }

  if (verbose) {
    cat("Matriz aumentada inicial (A|b):\n")
    pretty_print(Aug); cat("\n")
  }

  # Eliminação com pivoteamento parcial
  for (k in 1:(n-1)) {
    # encontrar linha com maior |A[i,k]| para i = k..n
    pivot_row <- which.max(abs(Aug[k:n, k])) + (k - 1)
```

```
  if (pivot_row != k) {
    Aug[c(k, pivot_row), ] <- Aug[c(pivot_row, k), ]
    if (verbose) {
      cat(sprintf("Swap: linha %d <-> linha %d\n\n", k, pivot_row))
      pretty_print(Aug); cat("\n")
    }
  }

  # Eliminar abaixo do pivô
  for (i in (k+1):n) {
    if (abs(Aug[k, k]) < .Machine$double.eps) stop("Pivô (Aug[k,k]) é zero (ou muito peque
    m <- Aug[i, k] / Aug[k, k]
    cols <- k:(n+1)
    Aug[i, cols] <- Aug[i, cols] - m * Aug[k, cols]

    if (verbose) {
      cat(sprintf("Eliminação: usar linha %d para zerar coluna %d da linha %d (m = %s)\n\n
                  k, k, i, MASS::fractions(m)))
      pretty_print(Aug); cat("\n")
    }
  }
}

# Substituição regressiva (corrigida)
x <- numeric(n)
for (i in n:1) {
  if (i == n) {
    x[i] <- Aug[i, n + 1] / Aug[i, i]
  } else {
    if (i + 1 <= n) {
      x[i] <- (Aug[i, n + 1] - sum(Aug[i, (i + 1):n] * x[(i + 1):n])) / Aug[i, i]
    } else {
      x[i] <- Aug[i, n + 1] / Aug[i, i]
    }
  }
}

if (verbose) {
  cat("Matriz triangular superior final (A|b):\n")
  pretty_print(Aug); cat("\n")
  cat("Solução x:\n")
  pretty_print(x); cat("\n")
```

```r
  }

  return(list(x = x, Aug = Aug))
}

# --- Teste ---
A <- matrix(c(2,2,1,1,
              1,-1,2,-1,
              3,2,-3,-2,
              4,3,2,1), nrow=4, byrow=TRUE)
b <- c(7,1,4,12)

res <- gauss_parcial(A, b, verbose = TRUE)
```

```
Matriz aumentada inicial (A|b):
                 b
[1,]  2  2  1  1  7
[2,]  1 -1  2 -1  1
[3,]  3  2 -3 -2  4
[4,]  4  3  2  1 12

Swap: linha 1 <-> linha 4

                 b
[1,]  4  3  2  1 12
[2,]  1 -1  2 -1  1
[3,]  3  2 -3 -2  4
[4,]  2  2  1  1  7

Eliminação: usar linha 1 para zerar coluna 1 da linha 2 (m = 1/4)

                        b
[1,]    4    3    2    1   12
[2,]    0 -7/4  3/2 -5/4   -2
[3,]    3    2   -3   -2    4
[4,]    2    2    1    1    7

Eliminação: usar linha 1 para zerar coluna 1 da linha 3 (m = 3/4)

                           b
[1,]     4     3     2     1    12
[2,]     0  -7/4   3/2  -5/4    -2
```

```
[3,]     0  -1/4  -9/2 -11/4    -5
[4,]     2     2     1     1     7

Eliminação: usar linha 1 para zerar coluna 1 da linha 4 (m = 1/2)

                            b
[1,]     4     3     2     1    12
[2,]     0  -7/4   3/2  -5/4    -2
[3,]     0  -1/4  -9/2 -11/4    -5
[4,]     0   1/2     0   1/2     1

Eliminação: usar linha 2 para zerar coluna 2 da linha 3 (m = 1/7)

                            b
[1,]     4     3     2     1    12
[2,]     0  -7/4   3/2  -5/4    -2
[3,]     0     0 -33/7 -18/7 -33/7
[4,]     0   1/2     0   1/2     1

Eliminação: usar linha 2 para zerar coluna 2 da linha 4 (m = -2/7)

                            b
[1,]     4     3     2     1    12
[2,]     0  -7/4   3/2  -5/4    -2
[3,]     0     0 -33/7 -18/7 -33/7
[4,]     0     0   3/7   1/7   3/7

Eliminação: usar linha 3 para zerar coluna 3 da linha 4 (m = -1/11)

                            b
[1,]     4     3     2     1    12
[2,]     0  -7/4   3/2  -5/4    -2
[3,]     0     0 -33/7 -18/7 -33/7
[4,]     0     0     0 -1/11     0

Matriz triangular superior final (A|b):
                            b
[1,]     4     3     2     1    12
[2,]     0  -7/4   3/2  -5/4    -2
[3,]     0     0 -33/7 -18/7 -33/7
[4,]     0     0     0 -1/11     0

Solução x:
```

```
[1] 1 2 1 0
```

4.2 Matrizes de Hilbert

```r
# Geração de Hilbert e resolução
gauss_hilbert <- function(A, b) {
  n <- nrow(A)
  Aug <- cbind(A, b)   # matriz aumentada (n x (n+1))

  # Eliminação com pivoteamento parcial
  for (k in 1:(n-1)) {
    # encontrar linha com maior |A[i,k]| para i = k..n
    pivot_row <- which.max(abs(Aug[k:n, k])) + (k - 1)
    if (pivot_row != k) {
      Aug[c(k, pivot_row), ] <- Aug[c(pivot_row, k), ]
    }

    # Eliminar abaixo do pivô
    for (i in (k+1):n) {
      if (abs(Aug[k, k]) < .Machine$double.eps) stop("Pivô (Aug[k,k]) é zero (ou muito peque
      m <- Aug[i, k] / Aug[k, k]
      cols <- k:(n+1)
      Aug[i, cols] <- Aug[i, cols] - m * Aug[k, cols]
    }
  }

  # Substituição regressiva
  x <- numeric(n)
  for (i in n:1) {
    if (i == n) {
      x[i] <- Aug[i, n + 1] / Aug[i, i]
    } else {
      x[i] <- (Aug[i, n + 1] - sum(Aug[i, (i + 1):n] * x[(i + 1):n])) / Aug[i, i]
    }
  }

  return(list(x = x, Aug = Aug))
}

# Função para gerar matriz de Hilbert
hilbert_matrix <- function(n) {
```

```
  H <- matrix(0, n, n)
  for (i in 1:n) {
    for (j in 1:n) {
      H[i, j] <- 1 / (i + j - 1)
    }
  }
  return(H)
}

# Resolver sistemas com matrizes de Hilbert para n = 4, 10, 20
ns <- c(4, 10, 20)
results <- list()

for (n in ns) {
  cat("=== MATRIZ DE HILBERT n =", n, "===\n")

  # Gerar matriz de Hilbert
  H <- hilbert_matrix(n)

  # Gerar vetor b (soma das linhas)
  b <- numeric(n)
  for (i in 1:n) {
    b[i] <- sum(H[i, ])
  }

  # Resolver sistema
  tryCatch({
    res <- gauss_hilbert(H, b)

    cat("Solução encontrada x:\n")
    print(res$x)

    # Calcular erro
    sol_exata <- rep(1, n)
    erro <- max(abs(res$x - sol_exata))
    cat("Erro máximo (||x - x*||∞):", erro, "\n")

    # Número de condição da matriz
    cond_num <- kappa(H)
    cat("Número de condição da matriz:", cond_num, "\n")

    results[[as.character(n)]] <- list(
```

```
      x = res$x,
      erro = erro,
      cond_num = cond_num
    )

  }, error = function(e) {
    cat("ERRO ao resolver sistema para n =", n, ":", e$message, "\n")
  })

  cat("\n", strrep("-", 60), "\n\n")
}
```

```
=== MATRIZ DE HILBERT n = 4 ===
Solução encontrada x:
[1] 1.000000000000019 0.999999999999802 1.000000000000456 0.999999999999711
Erro máximo (||x - x*||∞): 4.55857573911089e-13
Número de condição da matriz: 20093.9251921593

 ------------------------------------------------------------

=== MATRIZ DE HILBERT n = 10 ===
Solução encontrada x:
 [1] 1.000000002084221 0.999999822163300 1.000003752771577 0.999966128092409
 [5] 1.000160636418172 0.999560486944131 1.000718271450449 0.999308193820036
 [9] 1.000362146740079 0.999920558181244
Erro máximo (||x - x*||∞): 0.000718271450449004
Número de condição da matriz: 24161803237989.7

 ------------------------------------------------------------

=== MATRIZ DE HILBERT n = 20 ===
ERRO ao resolver sistema para n = 20 : Pivô (Aug[k,k]) é zero (ou muito pequeno).

 ------------------------------------------------------------
```

4.3 Gauss-Jacobi

```
# Método de Gauss-Jacobi
gauss_jacobi <- function(A, b, x0 = NULL, tol = 1e-6, max_iter = 1000, verbose = TRUE) {
  n <- nrow(A)
```

```r
# Verificar critério das linhas RIGOROSAMENTE
cat("=== VERIFICAÇÃO DO CRITÉRIO DAS LINHAS ===\n")
for (i in 1:n) {
  soma_fora_diagonal <- sum(abs(A[i, -i]))  # soma dos |a_ij| onde j   i
  alpha_i <- soma_fora_diagonal / abs(A[i, i])

  cat(sprintf("Linha %d:   = %.4f", i, alpha_i))

  if (alpha_i >= 1) {
    cat(" →  NÃO SATISFEITO\n")
    stop(sprintf("CRITÉRIO DAS LINHAS FALHOU na linha %d (  = %.4f   1). Execução interrompi
  } else {
    cat(" →  OK\n")
  }
}

cat("  Critério das linhas SATISFEITO para todas as linhas\n")
cat("  máximo =", max(sapply(1:n, function(i) sum(abs(A[i, -i])) / abs(A[i, i]))), "\n\n")

# Inicialização
if (is.null(x0)) {
  x0 <- rep(0, n)
}
x <- x0
x_new <- numeric(n)

# Iterações
for (iter in 1:max_iter) {
  for (i in 1:n) {
    soma <- b[i]
    for (j in 1:n) {
      if (j != i) {
        soma <- soma - A[i, j] * x[j]
      }
    }
    x_new[i] <- soma / A[i, i]
  }

  # Critério de parada: norma da diferença
  diff_norm <- max(abs(x_new - x))

  if (verbose && iter %% 10 == 0) {
```

```
      cat(sprintf("Iteração %d: ||x_new - x|| = %e\n", iter, diff_norm))
    }

    if (diff_norm < tol) {
      if (verbose) {
        cat(sprintf("\n Convergência alcançada na iteração %d\n", iter))
        cat("Solução final:\n")
        print(x_new)
      }
      return(list(x = x_new, iter = iter, converged = TRUE))
    }

    x <- x_new
  }

  warning("Número máximo de iterações atingido")
  return(list(x = x_new, iter = max_iter, converged = FALSE))
}

# Função auxiliar para testar múltiplos sistemas
testar_sistema <- function(A, b, nome_sistema) {
  cat(sprintf("\n%s\n", strrep("=", 60)))
  cat(sprintf("TESTANDO: %s\n", nome_sistema))
  cat(sprintf("%s\n", strrep("=", 60)))

  tryCatch({
    resultado <- gauss_jacobi(A, b, tol = 1e-6, max_iter = 200, verbose = TRUE)
    cat(" SUCESSO: Método convergiu\n")
    return(resultado)
  }, error = function(e) {
    cat(" FALHA:", e$message, "\n")
    return(NULL)
  })
}
```

```
# Sistema do item 3a
# Sistema do item 3a
A1 <- matrix(c(2, 2, 1, 1,
               1, -1, 2, -1,
               3, 2, -3, -2,
               4, 3, 2, 1), nrow = 4, byrow = TRUE)
b1 <- c(7, 1, 4, 12)
```

```r
# Testar sistema original
result_3a <- testar_sistema(A1, b1, "Sistema 3a - Original")
```

```
============================================================
TESTANDO: Sistema 3a - Original
============================================================
=== VERIFICAÇÃO DO CRITÉRIO DAS LINHAS ===
Linha 1:   = 2.0000 →  NÃO SATISFEITO
  FALHA: CRITÉRIO DAS LINHAS FALHOU na linha 1 (  = 2.0000   1). Execução interrompida.
```

```r
# Resolver sistemas de Hilbert com verificação rigorosa
results_hilbert <- list()

for (n in c(4, 10, 20)) {
  cat(sprintf("\n\n%s\n", strrep("=", 60)))
  cat(sprintf("MATRIZ DE HILBERT n = %d\n", n))
  cat(sprintf("%s\n", strrep("=", 60)))

  # Gerar matriz de Hilbert
  H <- hilbert_matrix(n)

  # Gerar vetor b (soma das linhas)
  b <- numeric(n)
  for (i in 1:n) {
    b[i] <- sum(H[i, ])
  }

  # Testar sistema
  resultado <- testar_sistema(H, b, sprintf("Matriz de Hilbert n = %d", n))

  if (!is.null(resultado)) {
    # Calcular erro
    sol_exata <- rep(1, n)
    erro <- max(abs(resultado$x - sol_exata))

    results_hilbert[[as.character(n)]] <- list(
      x = resultado$x,
      erro = erro,
      iter = resultado$iter,
      converged = resultado$converged
```

```
    )

    cat(sprintf("Erro máximo: %.2e\n", erro))
  }
}
```

```
============================================================
MATRIZ DE HILBERT n = 4
============================================================

============================================================
TESTANDO: Matriz de Hilbert n = 4
============================================================
=== VERIFICAÇÃO DO CRITÉRIO DAS LINHAS ===
Linha 1:   = 1.0833 →   NÃO SATISFEITO
  FALHA: CRITÉRIO DAS LINHAS FALHOU na linha 1 (  = 1.0833   1). Execução interrompida.


============================================================
MATRIZ DE HILBERT n = 10
============================================================

============================================================
TESTANDO: Matriz de Hilbert n = 10
============================================================
=== VERIFICAÇÃO DO CRITÉRIO DAS LINHAS ===
Linha 1:   = 1.9290 →   NÃO SATISFEITO
  FALHA: CRITÉRIO DAS LINHAS FALHOU na linha 1 (  = 1.9290   1). Execução interrompida.


============================================================
MATRIZ DE HILBERT n = 20
============================================================

============================================================
TESTANDO: Matriz de Hilbert n = 20
============================================================
=== VERIFICAÇÃO DO CRITÉRIO DAS LINHAS ===
Linha 1:   = 2.5977 →   NÃO SATISFEITO
  FALHA: CRITÉRIO DAS LINHAS FALHOU na linha 1 (  = 2.5977   1). Execução interrompida.
```

4.4 Discussão

Compare Jacobi vs. Gauss-Seidel e discuta condicionamento.

5 Questão 4 – Sistemas Não Lineares

5.1 Implementação do Método de Newton

```
metodo_newton <- function(F, J, x0, epsilon, max_iter) {
  x <- x0
  iter <- 0
  norm_f <- norm(matrix(F(x)), "2")
  norm_dx <- Inf

  cat("Método de Newton para Sistemas Não Lineares:\n")
  cat("Iter |     x1     |     x2     |  ||F(x)||  | ||Δx||∞ \n")
  cat("-------------------------------------------------------\n")

  while ((norm_f >= epsilon || norm_dx >= epsilon) && iter < max_iter) {
    iter <- iter + 1
    x_old <- x

    # Passo 4: Calcular s = -J(x)^{-1}F(x)
    delta_x <- solve(J(x), -F(x))

    # Passo 5: Atualizar x
    x <- x + delta_x

    # Calcular normas para critérios de parada
    norm_f <- norm(matrix(F(x)), "2")
    norm_dx <- max(abs(x - x_old))

    cat(sprintf("%4d | %10.6f | %10.6f | %9.2e | %9.2e\n",
                iter, x[1], x[2], norm_f, norm_dx))

    # Verificar ambos os critérios de parada
    if (norm_f < epsilon && norm_dx < epsilon) {
      break
    }
  }
```

```r
  return(list(x = x, iter = iter, norm_f = norm_f, norm_dx = norm_dx))
}

# Definir o sistema não linear da questão
F <- function(x) {
  c(4*x[1] - x[1]^3 + x[2],
    (-x[1]^2)/9 + (4*x[2] - x[2]^2)/4 + 1)
}

# Definir a Jacobiana
J <- function(x) {
  matrix(c(4 - 3*x[1]^2, 1,
           -2*x[1]/9, (4 - 2*x[2])/4),
         nrow = 2, byrow = TRUE)
}

# Executar método
x0 <- c(0, 0)  # Ponto inicial
epsilon <- 1e-4
max_iter <- 100

result <- metodo_newton(F, J, x0, epsilon, max_iter)
```

```
Método de Newton para Sistemas Não Lineares:
Iter |     x1     |     x2     |  ||F(x)||  | ||Δx||∞
-------------------------------------------------------
   1 |   0.250000 |  -1.000000 |  2.57e-01 |  1.00e+00
   2 |   0.209561 |  -0.830201 |  7.48e-03 |  1.70e-01
   3 |   0.208520 |  -0.825014 |  6.88e-06 |  5.19e-03
   4 |   0.208519 |  -0.825009 |  5.96e-12 |  4.81e-06
```

```r
cat(sprintf("\nSolução: x = [%.6f, %.6f]\n", result$x[1], result$x[2]))
```

```
Solução: x = [0.208519, -0.825009]
```

```r
cat(sprintf("Iterações: %d\n", result$iter))
```

```
Iterações: 4
```

```r
cat(sprintf("||F(x)||  = %.2e\n", result$norm_f))
```

```
||F(x)||  = 5.96e-12
```

```r
cat(sprintf("||Δx||∞ = %.2e\n", result$norm_dx))
```

```
||Δx||∞ = 4.81e-06
```

5.2 Resultados e Gráficos

```r
library(ggplot2)

# Criar grid de pontos
x1 <- seq(-5, 5, length.out = 200)
x2 <- seq(-5, 5, length.out = 200)
grid <- expand.grid(x1 = x1, x2 = x2)

# Calcular funções do sistema CORRETO (Questão 4a)
grid$F1 <- 4*grid$x1 - grid$x1^3 + grid$x2
grid$F2 <- - (grid$x1^2)/4 + 4*grid$x2 - grid$x2^2 + 1

# Solução encontrada no item 4a
solucao <- c(0.208519, -0.825009)

# Gráfico do sistema com cores harmoniosas
ggplot(grid, aes(x = x1, y = x2)) +
  # Curva da primeira equação F1(x) = 0
  geom_contour(aes(z = F1), breaks = 0, color = "#d6277e", size = 1.2) +
  # Curva da segunda equação F2(x) = 0
  geom_contour(aes(z = F2), breaks = 0, color = "purple", size = 1.2) +

  # Marcar a solução encontrada
  annotate("point", x = solucao[1], y = solucao[2],
           color = "black", size = 2, fill = "black", shape = 21, stroke = 2) +
  annotate("text", x = solucao[1] - 1, y = solucao[2],
           label = sprintf("solução", solucao[1], solucao[2]),
           color = "black", size = 4, fontface = "bold") +

  # Marcar ponto inicial usado
```

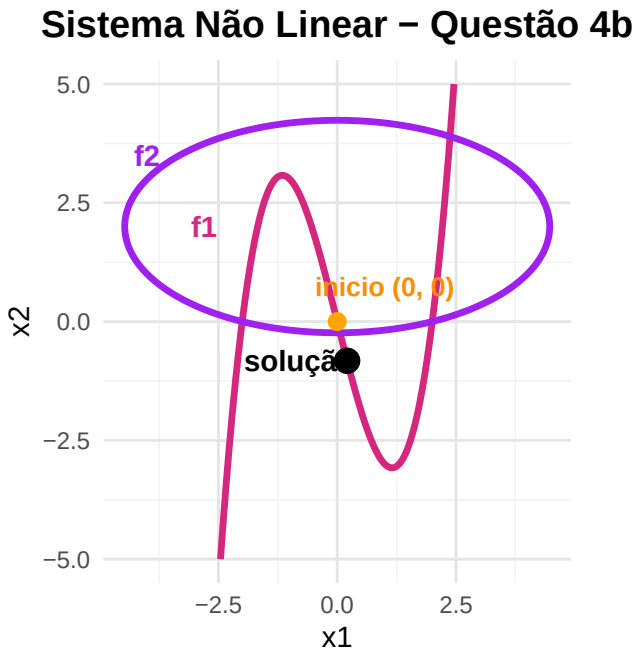
```
  annotate("point", x = 0, y = 0,
           color = "orange", size = 3, shape = 16) +
  annotate("text", x = 0 + 1, y = 0 + 0.75,
           label = "inicio (0, 0)",
           color = "darkorange", size = 3.5, fontface = "bold") +

  # Adicionar legenda das curvas
  annotate("text", x = -2.8, y = 2,
           label = "f1", color = "#d6277e", size = 4, fontface = "bold") +
  annotate("text", x = -4, y = 3.5,
           label = "f2", color = "purple", size = 4, fontface = "bold") +

  labs(x = "x1", y = "x2",
       title = "Sistema Não Linear - Questão 4b") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
    plot.subtitle = element_text(hjust = 0.5, size = 11),
    panel.grid.major = element_line(color = "grey90"),
    panel.grid.minor = element_line(color = "grey95")
  ) +
  coord_fixed(ratio = 1)
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```

```
Warning in sprintf("solução", solucao[1], solucao[2]): 2 arguments not used by
format 'solução'
```




```r
# Executar método
metodo_newton <- function(F, J, x0, epsilon, max_iter) {
  x <- x0
  iter <- 0
  norm_f <- norm(matrix(F(x)), "2")
  norm_dx <- Inf

  cat("Método de Newton para Sistemas Não Lineares:\n")
  cat("Iter |     x1     |     x2     |  ||F(x)||  | ||Δx||∞ \n")
  cat("-----------------------------------------------------------\n")

  while ((norm_f >= epsilon || norm_dx >= epsilon) && iter < max_iter) {
    iter <- iter + 1
    x_old <- x

    # Passo 4: Calcular s = -J(x)^{-1}F(x)
    delta_x <- solve(J(x), -F(x))

    # Passo 5: Atualizar x
    x <- x + delta_x

    # Calcular normas para critérios de parada
    norm_f <- norm(matrix(F(x)), "2")
```

```
    norm_dx <- max(abs(x - x_old))

    cat(sprintf("%4d | %10.6f | %10.6f | %9.2e | %9.2e\n",
                iter, x[1], x[2], norm_f, norm_dx))

    # Verificar ambos os critérios de parada
    if (norm_f < epsilon && norm_dx < epsilon) {
      break
    }
  }

  return(list(x = x, iter = iter, norm_f = norm_f, norm_dx = norm_dx))
}

# Definir o sistema não linear da questão
F <- function(x) {
  c(4*x[1] - x[1]^3 + x[2],
    (-x[1]^2)/9 + (4*x[2] - x[2]^2)/4 + 1)
}

# Definir a Jacobiana
J <- function(x) {
  matrix(c(4 - 3*x[1]^2, 1,
           -2*x[1]/9, (4 - 2*x[2])/4),
         nrow = 2, byrow = TRUE)
}

# Executar método
x0 <- c(0, 0)  # Ponto inicial
epsilon <- 1e-12
max_iter <- 100

result <- metodo_newton(F, J, x0, epsilon, max_iter)
```

```
Método de Newton para Sistemas Não Lineares:
Iter |     x1     |     x2     |  ||F(x)||  | ||Δx||∞
-------------------------------------------------------
   1 |   0.250000 |  -1.000000 |  2.57e-01 |  1.00e+00
   2 |   0.209561 |  -0.830201 |  7.48e-03 |  1.70e-01
   3 |   0.208520 |  -0.825014 |  6.88e-06 |  5.19e-03
   4 |   0.208519 |  -0.825009 |  5.96e-12 |  4.81e-06
   5 |   0.208519 |  -0.825009 |  1.11e-16 |  4.16e-12
```

```
   6 |   0.208519 |  -0.825009 |  0.00e+00 |  1.11e-16
```

```r
cat(sprintf("\nSolução: x = [%.6f, %.6f]\n", result$x[1], result$x[2]))
```

```
Solução: x = [0.208519, -0.825009]
```

```r
cat(sprintf("Iterações: %d\n", result$iter))
```

```
Iterações: 6
```

```r
cat(sprintf("||F(x)||  = %.2e\n", result$norm_f))
```

```
||F(x)||  = 0.00e+00
```

```r
cat(sprintf("||Δx||ϖ = %.2e\n", result$norm_dx))
```

```
||Δx||ϖ = 1.11e-16
```

```r
# Executar método
metodo_newton <- function(F, J, x0, epsilon, max_iter) {
  x <- x0
  iter <- 0
  norm_f <- norm(matrix(F(x)), "2")
  norm_dx <- Inf

  cat("Método de Newton para Sistemas Não Lineares:\n")
  cat("Iter |     x1     |     x2     |  ||F(x)||  | ||Δx||ϖ \n")
  cat("-------------------------------------------------------------\n")

  while ((norm_f >= epsilon || norm_dx >= epsilon) && iter < max_iter) {
    iter <- iter + 1
    x_old <- x

    # Passo 4: Calcular s = -J(x)^{-1}F(x)
    delta_x <- solve(J(x), -F(x))

    # Passo 5: Atualizar x
    x <- x + delta_x
```

```
    # Calcular normas para critérios de parada
    norm_f <- norm(matrix(F(x)), "2")
    norm_dx <- max(abs(x - x_old))

    cat(sprintf("%4d | %10.6f | %10.6f | %9.2e | %9.2e\n",
                iter, x[1], x[2], norm_f, norm_dx))

    # Verificar ambos os critérios de parada
    if (norm_f < epsilon && norm_dx < epsilon) {
      break
    }
  }

  return(list(x = x, iter = iter, norm_f = norm_f, norm_dx = norm_dx))
}

# Definir o sistema não linear da questão
F <- function(x) {
  c(4*x[1] - x[1]^3 + x[2],
    (-x[1]^2)/9 + (4*x[2] - x[2]^2)/4 + 1)
}

# Definir a Jacobiana
J <- function(x) {
  matrix(c(4 - 3*x[1]^2, 1,
           -2*x[1]/9, (4 - 2*x[2])/4),
         nrow = 2, byrow = TRUE)
}

# Executar método
x0 <- c(0, 0)  # Ponto inicial
epsilon <- 1e-25
max_iter <- 100

result <- metodo_newton(F, J, x0, epsilon, max_iter)
```

```
Método de Newton para Sistemas Não Lineares:
Iter |     x1     |     x2     |  ||F(x)||  | ||Δx||∞
-------------------------------------------------------
   1 |   0.250000 |  -1.000000 |  2.57e-01 |  1.00e+00
   2 |   0.209561 |  -0.830201 |  7.48e-03 |  1.70e-01
```

```
   3 |   0.208520 |  -0.825014 |  6.88e-06 |  5.19e-03
   4 |   0.208519 |  -0.825009 |  5.96e-12 |  4.81e-06
   5 |   0.208519 |  -0.825009 |  1.11e-16 |  4.16e-12
   6 |   0.208519 |  -0.825009 |  0.00e+00 |  1.11e-16
   7 |   0.208519 |  -0.825009 |  0.00e+00 |  0.00e+00
```

```
cat(sprintf("\nSolução: x = [%.6f, %.6f]\n", result$x[1], result$x[2]))
```

```
Solução: x = [0.208519, -0.825009]
```

```
cat(sprintf("Iterações: %d\n", result$iter))
```

```
Iterações: 7
```

```
cat(sprintf("||F(x)||  = %.2e\n", result$norm_f))
```

```
||F(x)||  = 0.00e+00
```

```
cat(sprintf("||Δx||∞ = %.2e\n", result$norm_dx))
```

```
||Δx||∞ = 0.00e+00
```

Discuta número de iterações, dependência do ponto inicial e tolerância.

6 Conclusão

Sintetize os resultados, discuta estabilidade numérica e condicionamento.

7 Bibliografia

- Slides de Aula
- Burden & Faires – *Análise Numérica*
- Chapra & Canale – *Métodos Numéricos para Engenharia*
- Documentação R e pacotes utilizados